

Representing ORM in Description Logic

Mustafa Jarrar

Birzeit University
mjarrar@birzeit.edu
www.jarrar.info



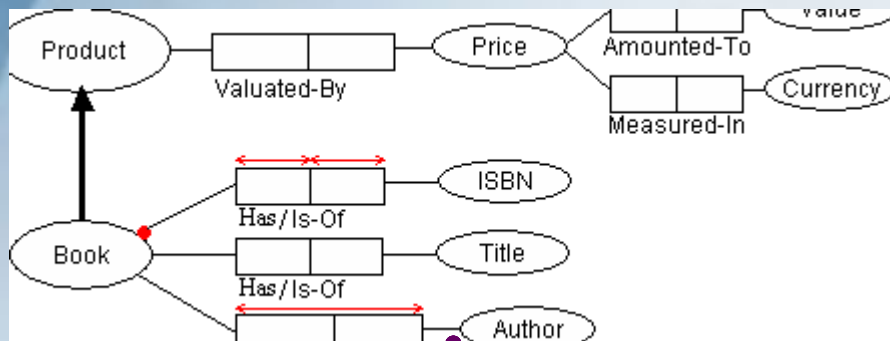
This lecture is based on

1. Mustafa Jarrar: Mapping ORM Into The SHOIN/OWL Description Logic- Towards A Methodological And Expressive Graphical Notation For Ontology Engineering . In OTM 2007 workshops: Proceedings of the International Workshop on Object-Role Modeling (ORM'07). Pages (729-741), LNCS 4805, Springer. ISBN: 9783540768890. Portugal. November, 2007
1. Mustafa Jarrar: Towards Automated Reasoning On ORM Schemes -Mapping ORM Into The DLR_idf Description Logic. In proceedings of the 26th International Conference on Conceptual Modeling (ER 2007). Pages (181-197). LNCS 4801, Springer. Auckland, New Zealand. ISBN 9783540755623. November 2007

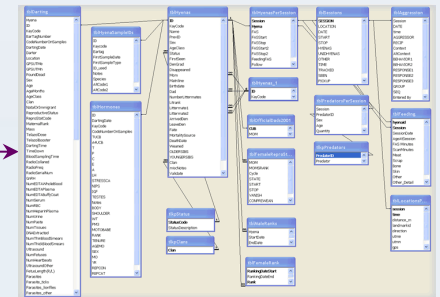
Outline

- Part I: Introduction (Why Description logics)
- Part II: Introduction to Description logics
- Part III: Mapping ORM into Description logic
- Something to think about 😊

Remember: ORM has many Applications



Originally



Database

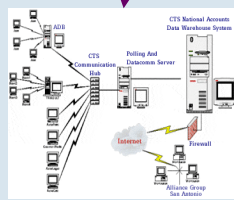
Later



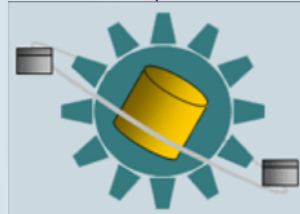
Ontology



XML Schema



Warehouse



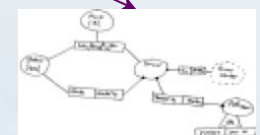
Business Rules



Web (x)Forms

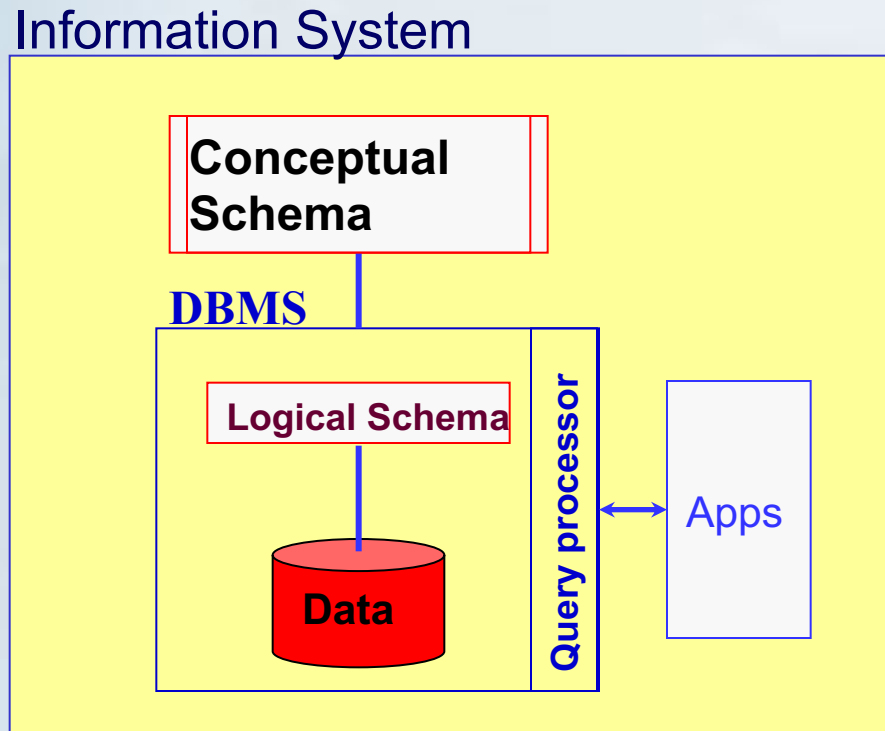


Requirements Engineering



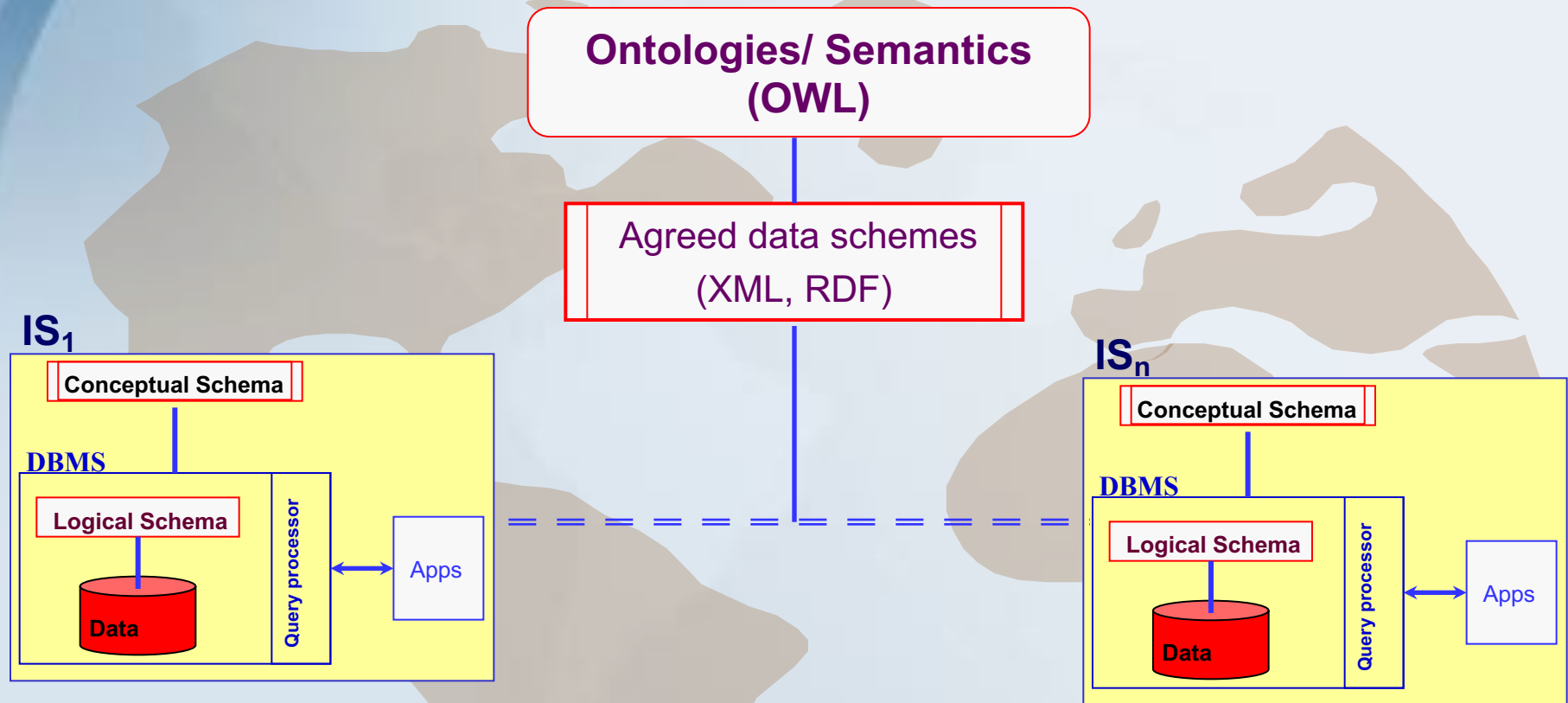
Record my recipes !

An Information System



- Each Information System is made for one organization.
- Why do we need conceptual schemes? for designing Information systems at the conceptual level.

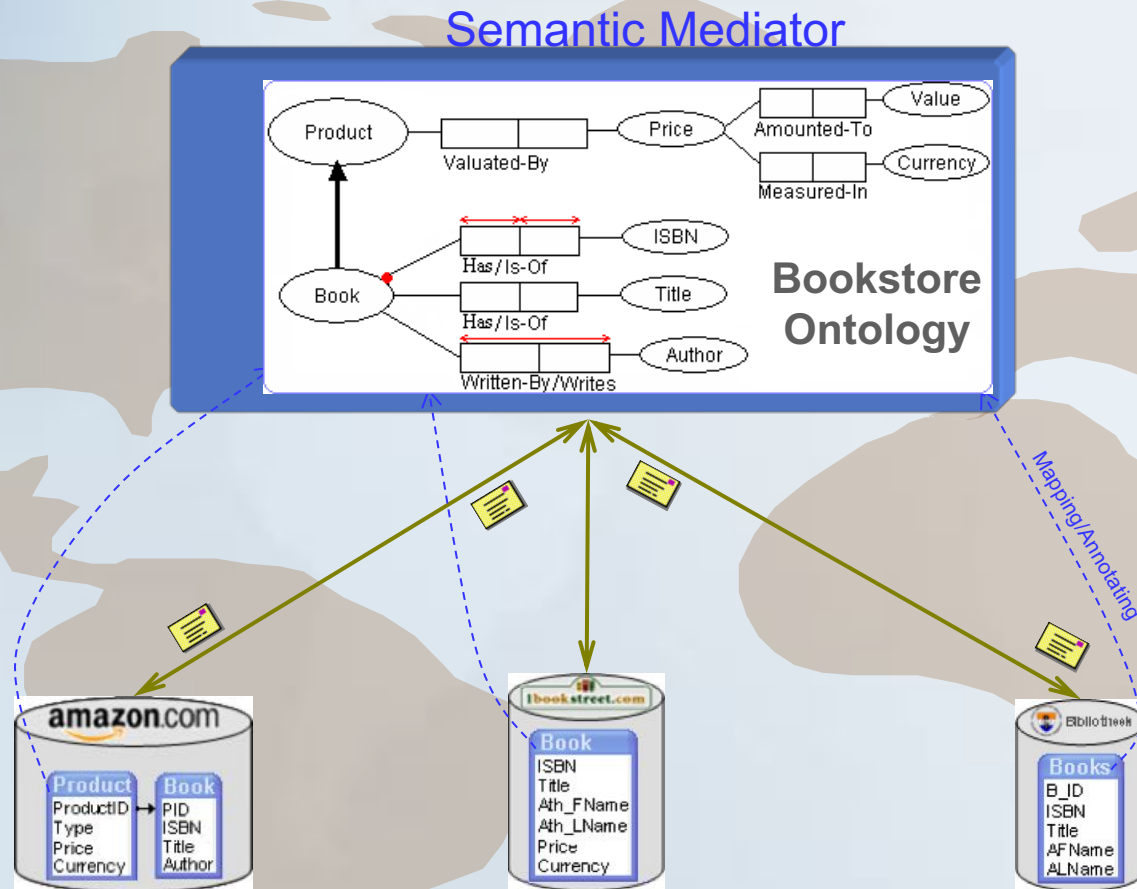
(Web) Information Systems



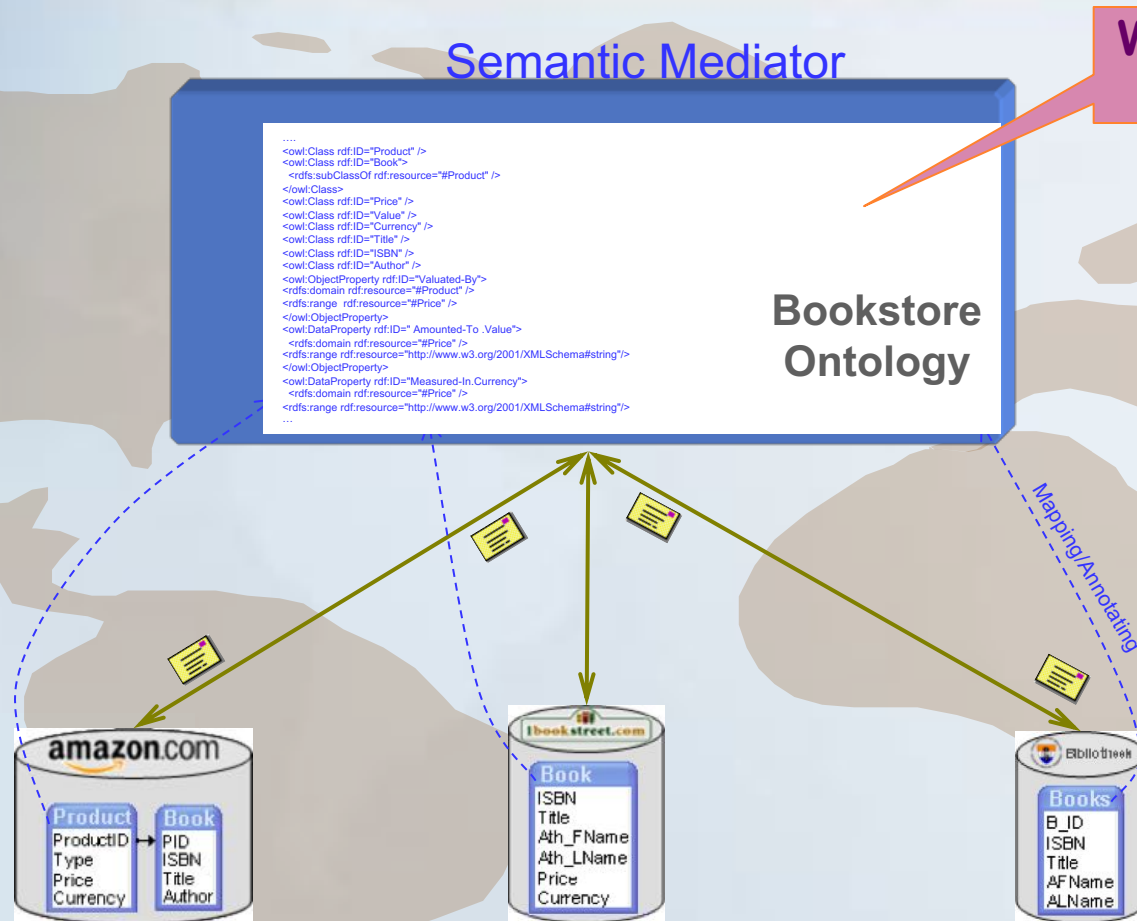
New needs:

Open data exchange, inter-organizational transactions, global queries...

(Web) Information Systems



(Web) Information Systems



OWL

- The Web Ontology Language (W3C Standard)
- Based on Description logic

```
....
<owl:Class rdf:ID="Product" />
<owl:Class rdf:ID="Book">
  <rdfs:subClassOf rdf:resource="#Product" />
</owl:Class>
<owl:Class rdf:ID="Price" />
<owl:Class rdf:ID="Value" />
<owl:Class rdf:ID="Currency" />
<owl:Class rdf:ID="Title" />
<owl:Class rdf:ID="ISBN" />
<owl:Class rdf:ID="Author" />
<owl:ObjectProperty rdf:ID="Valuated-By">
  <rdfs:domain rdf:resource="#Product" />
  <rdfs:range rdf:resource="#Price" />
</owl:ObjectProperty>
<owl:DataProperty rdf:ID="Amounted-To .Value">
  <rdfs:domain rdf:resource="#Price" />
  <rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#string"/>
</owl:ObjectProperty>
...
```

➔ Can we use ORM to Model OWL ontologies?

First Order Logic (FOL)

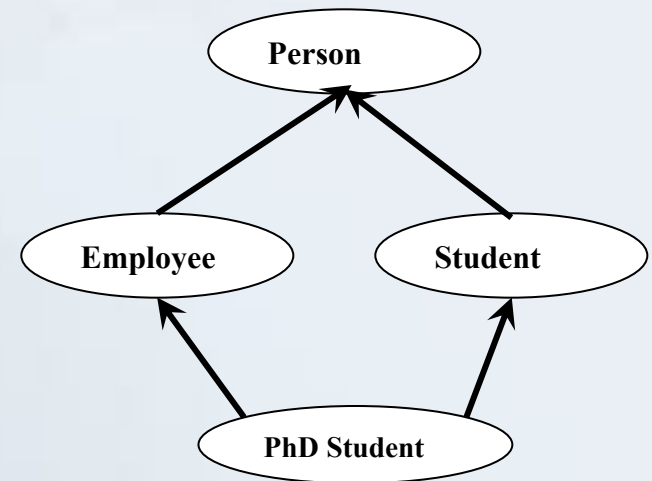
- FOL allows us to represent knowledge precisely (Syntax and Semantics).

$\forall x \text{ Employee}(x) \rightarrow \text{Person}(x)$

$\forall x \text{ Student}(x) \rightarrow \text{Person}(x)$

$\forall x \text{ PhDStudent}(x) \rightarrow \text{Student}(x)$

$\forall x \text{ PhDStudent}(x) \rightarrow \text{Employee}(x)$



- However, representation alone is not enough.
- We also need to process this knowledge and make use of it, i.e. Logical inference = (Reasoning).

First Order Logic (FOL)

Reasoning:

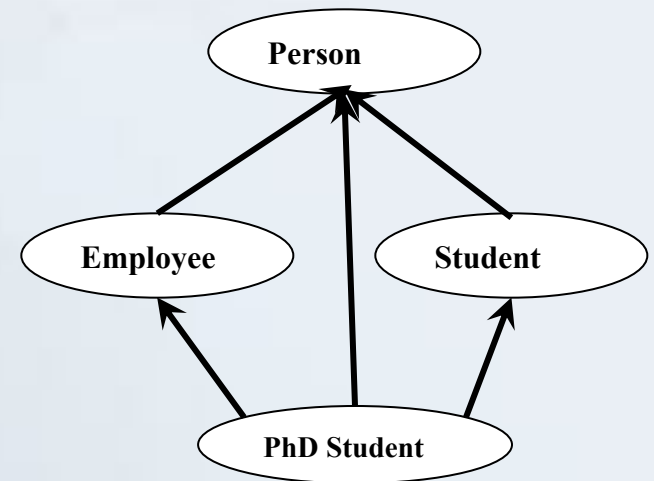
$\forall x \text{ Employee}(x) \rightarrow \text{Person}(x)$

$\forall x \text{ Student}(x) \rightarrow \text{Person}(x)$

$\forall x \text{ PhDStudent}(x) \rightarrow \text{Student}(x)$

$\forall x \text{ PhDStudent}(x) \rightarrow \text{Employee}(x)$

$\forall x \text{ PhDStudent}(x) \rightarrow \text{Person}(x)$



➔ How to process the above *axioms to know that an axiom can be derived from (=implied by) another axiom.*

First Order Logic (FOL)

Reasoning:

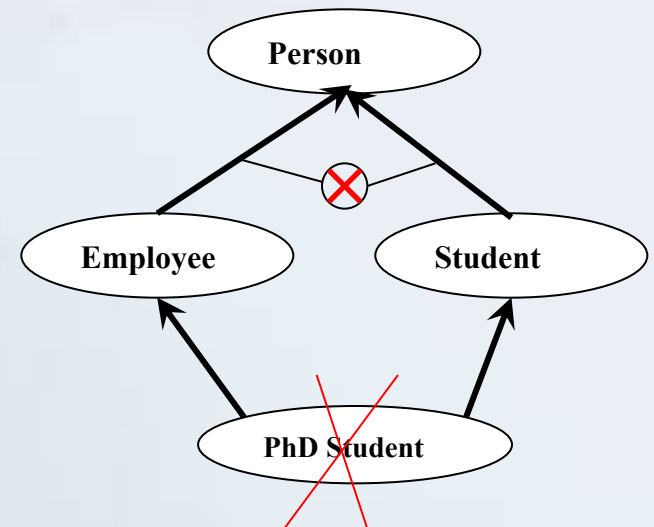
$\forall x \text{ Employee}(x) \rightarrow \text{Person}(x)$

$\forall x \text{ Student}(x) \rightarrow \text{Person}(x)$

$\forall x \text{ PhDStudent}(x) \rightarrow \text{Student}(x)$

$\forall x \text{ PhDStudent}(x) \rightarrow \text{Employee}(x)$

$\forall x \text{ Student}(x) \cap \text{Employee}(x) = \emptyset$



→ How to process the above *axioms* to know that an axiom can be derived from (=implied by) another axiom.

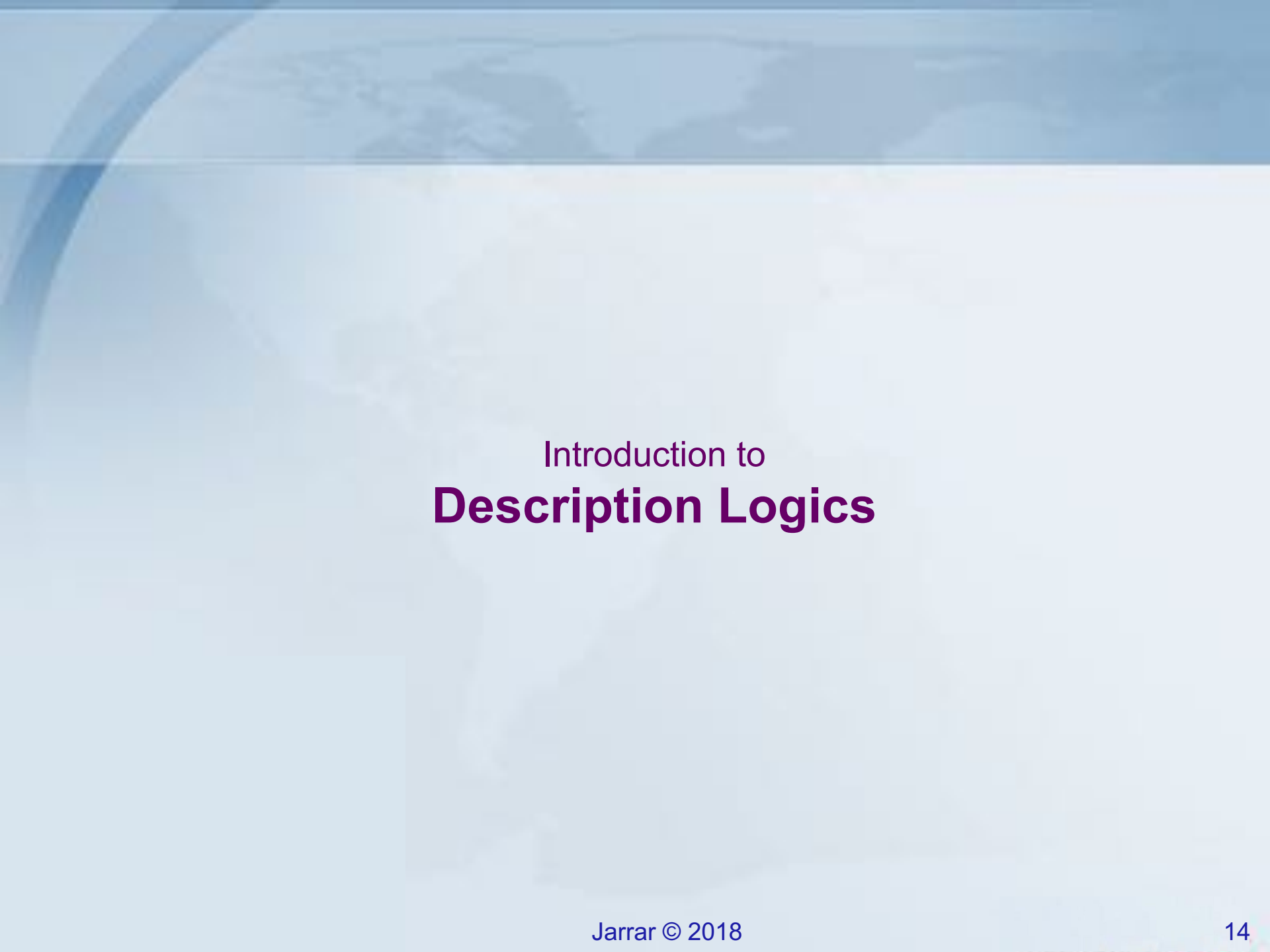
→ Find contradictions (satisfiability)

→ ...etc.

First Order Logic (FOL)

Reasoning on FOL is far too complex (i.e. not decidable)

→ Here comes description logics.

The background of the slide features a light blue, semi-transparent map of the world. A thick, dark blue arc curves from the top left towards the center of the frame. The text is centered on the map.

Introduction to **Description Logics**

Description Logics

- Description logics are a **family of logics** concerned with knowledge representation.
- A description logic is a **decidable** fragment of first-order logic, associated with a set of automatic **reasoning** procedures.
- The basic constructs for a description logic are the notion of a **concept** and the notion of a **relationship**.
- **Complex concept** and relationship expressions can be constructed from atomic concepts and relationships with suitable constructs between them.
- Example:
$$Mother \sqsubseteq Female \sqcap \exists HasChild$$
$$HumanMother \sqsubseteq Female \sqcap \exists HasChild.Person$$

Description Logics

Most known description logics are :

$\mathcal{FL}^-$ The simplest and less expressive description logic.
 $C, D \rightarrow A \mid C \sqcap D \mid \forall R.C \mid \exists R$

$\mathcal{AL}$ A more practical and expressive description logic.
 $C, D \rightarrow A \mid \top \mid \perp \mid \neg A \mid C \sqcap D \mid \forall R.C \mid \exists R.\top$

$\mathcal{SROIQ}$ Very popular description logic.
The logic underlying OWL.

$\mathcal{DLR}_{idf}$ Very expressive description logic,
Capable of representing most database constructs.

Description logic $\mathcal{AL}$

Syntax	Semantics
A	$A^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}$, A is a concept name
T	$T^{\mathcal{I}} = \Delta^{\mathcal{I}}$
$\perp$	$\perp^{\mathcal{I}} = \emptyset$
$\neg A$	$\Delta^{\mathcal{I}} \setminus A^{\mathcal{I}}$
$C \sqcap D$	$C^{\mathcal{I}} \cap D^{\mathcal{I}}$
$\forall R.C$	$\{x \in \Delta^{\mathcal{I}} \mid \forall y: (x,y) \in R^{\mathcal{I}} \Rightarrow y \in C^{\mathcal{I}}\}$
$\exists R.T$	$\{x \in \Delta^{\mathcal{I}} \mid \exists y \in \Delta^{\mathcal{I}}: (x,y) \in R^{\mathcal{I}}\}$
R	$R^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$, R is a role name

- Example constructs:

■ $\text{person} \sqcap \text{female}$

■ $\text{person} \sqcap \neg \text{female}$

■ $\text{person} \sqcap \exists \text{has_child}.T$

■ $\text{person} \sqcap \forall \text{has_child}.\perp$

More $\mathcal{AL}$ Family Members

- Disjunction ($\sqcup$)

$$C \sqcup D \qquad C^{\mathcal{I}} \cup D^{\mathcal{I}}$$

- Full existential quantification (ε)

$$\exists R.C \qquad \{x \in \Delta^{\mathcal{I}} \mid \exists y \in \Delta^{\mathcal{I}} : (x,y) \in R^{\mathcal{I}} \wedge y \in C^{\mathcal{I}}\}$$

- Number restrictions ($\mathcal{N}$)

$$\begin{array}{ll} \exists_{\geq n} R & \{x \in \Delta^{\mathcal{I}} \mid \|\{y \mid (x,y) \in R^{\mathcal{I}}\}\| \geq n\} \\ \exists_{\leq n} R & \{x \in \Delta^{\mathcal{I}} \mid \|\{y \mid (x,y) \in R^{\mathcal{I}}\}\| \leq n\} \end{array}$$

- Full negation ($\neg$)

$$\neg C \qquad \Delta^{\mathcal{I}} \setminus C^{\mathcal{I}}$$

- Example:

$$\text{person} \sqcap (\exists_{\leq 1} \text{has_child} \sqcup (\exists_{\geq 2} \text{has_child} \sqcap \exists \text{has_child.female}))$$

DL as fragments of Predicate Logic

- Any concept D as unary predicate with 1 free variable
- Any role R as primitive binary predicate
- $\exists R.C$ corresponds to
 $\exists y. R(x,y) \wedge C(y)$
- $\forall R.C$ corresponds to
 $\forall y. R(x,y) \Rightarrow C(y)$
- $\exists_{\neq n} R$ corresponds to
 $\exists y_1, \dots, y_n. R(x, y_1) \wedge \dots \wedge R(x, y_n) \wedge \forall i < j. y_i \neq y_j$
- $\exists_{=n} R$ corresponds to
 $\forall y_1, \dots, y_{n+1}. R(x, y_1) \wedge \dots \wedge R(x, y_{n+1}) \Rightarrow \exists i < j. y_i = y_j$
- Last two examples demonstrate advantage of variable-free syntax

DL Knowledge Base

- DL Knowledge Base (KB) normally separated into 2 parts:
 - TBox is a set of axioms describing structure of domain (i.e., a conceptual schema), e.g.:
 - $\text{HappyFather} \equiv \text{Man} \sqcap \exists \text{hasChild.Female} \sqcap \dots$
 - $\text{Elephant} \sqsubseteq \text{Animal} \sqcap \text{Large} \sqcap \text{Grey}$
 - $\text{transitive}(\text{ancestor})$
 - ABox is a set of axioms describing a concrete situation (data), e.g.:
 - $\text{John}:\text{HappyFather}$
 - $\langle \text{John}, \text{Mary} \rangle:\text{hasChild}$

Terminologies or TBoxes

- Statements about how concepts and roles are related to each other
- Terminology or **TBox** $\mathcal{T}$ is a set of axioms
 - interpretation $\mathcal{I}$ satisfies $\mathcal{T}$ if $\mathcal{I}$ satisfies all axioms in $\mathcal{T}$
- Terminological axioms
 - $C \sqsubseteq D$ ($R \sqsubseteq S$) $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$ ($R^{\mathcal{I}} \subseteq S^{\mathcal{I}}$)
 - C is subsumed by D or D subsumes C
 - $C \equiv D$ ($R \equiv S$) $C^{\mathcal{I}} = D^{\mathcal{I}}$ ($R^{\mathcal{I}} = S^{\mathcal{I}}$)
 - C is equivalent to D
- Definitions
 - if the left hand side of an axiom is a **name**
 - $\text{parent} \equiv \text{person} \sqcap \exists \text{has_child} \text{ person}$
 - $\text{fairy} \sqsubseteq \text{person}$

Inference Services

- What type of inferences are interesting?
 - satisfiability of (named) concepts
 - subsumption of (named) concepts
 - equivalence of (named) concepts
 - disjointness of (named) concepts
- Description of concept parent
 - $\text{parent} = \text{person} \sqcap \exists \text{has_child}.\text{person}$
- We add two concepts
 - $\text{woman} = \text{female} \sqcap \text{person}$
 - $\text{mother} = \text{female} \sqcap \text{parent}$

Inference services based on Satisfiability

- All concept inference services can be reduced to concept satisfiability
- We assume service $\text{sat}(C, \mathcal{T})$, C a concept, $\mathcal{T}$ a TBox
- $\text{subsumes}(C, D, \mathcal{T}) \equiv \neg \text{sat}(\neg C \sqcap D, \mathcal{T})$
 - $C \sqsupseteq D$ holds $\leftrightarrow \neg(C \sqcap \neg D)$ unsatisfiable $\leftrightarrow \neg C \sqcap D$ unsatisfiable
- $\text{equivalence}(C, D, \mathcal{T}) \equiv \text{subsumes}(C, D, \mathcal{T}) \wedge \text{subsumes}(D, C, \mathcal{T})$
- $\text{disjoint}(C, D, \mathcal{T}) \equiv \neg \text{sat}(C \sqcap D, \mathcal{T})$

Inference service: concept subsumption

```
parent  $\equiv$  person  $\sqcap$   $\exists$ has_child.person  
woman  $\equiv$  person  $\sqcap$  female  
mother  $\equiv$  parent  $\sqcap$  female
```

Is a Mother always a Woman?

Yes, Mother Subsumes Woman

Description logic reasoners offer the computation of a Subsumption (taxonomy) of all named concepts

Description Logic Reasoners

- For example:



- They offer reasoning services for multiple TBoxes and ABoxes.
- They run as background reasoning engines.
- They understand DIG, which is a simple protocol (based on HTTP) along with an XML Schema.
- **Example:** $Student \sqsubseteq Person$

```
<impliesc>
  <catom name="Student"/>
  <catom name="Person"/>
</impliesc>
```

The $\mathcal{SHOIN}$ Description Logic

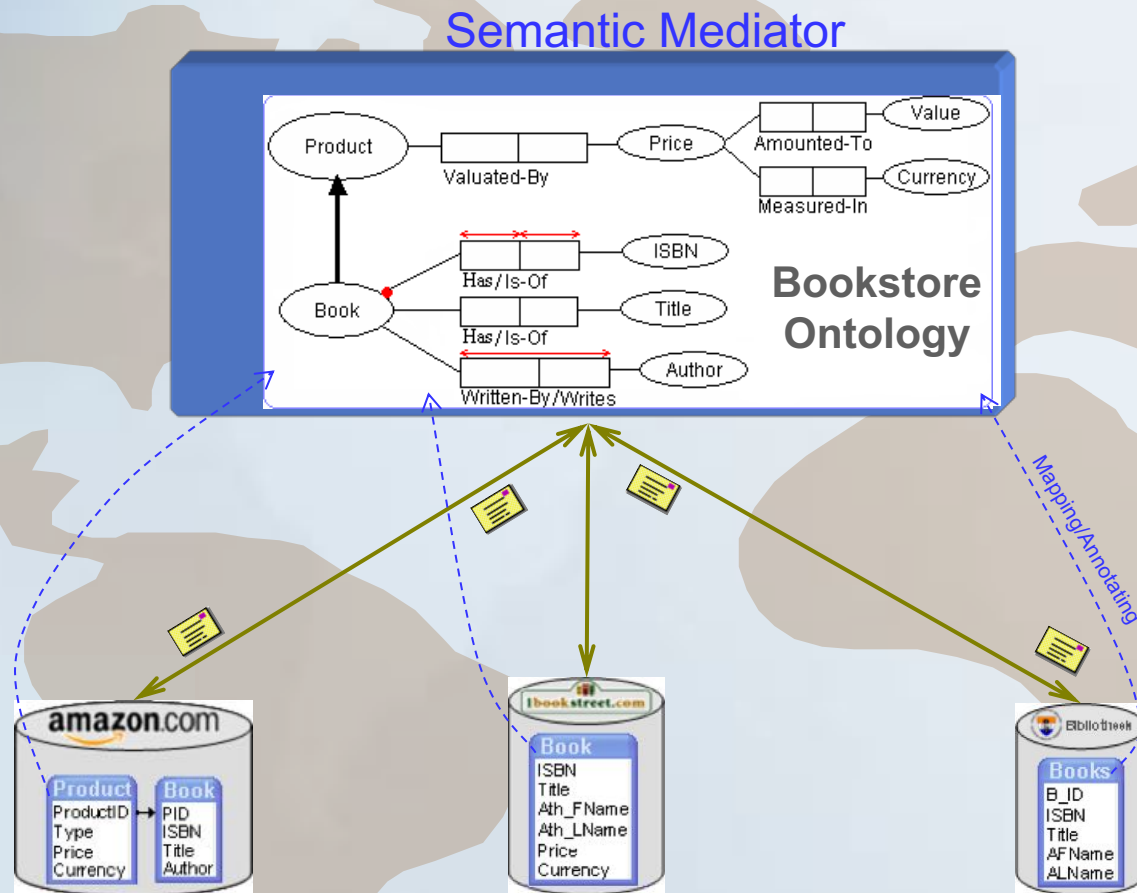
DL Syntax	Example	FOL Syntax
$C_1 \sqcap \dots \sqcap C_n$	Human $\sqcap$ Male	$C_1(x) \wedge \dots \wedge C_n(x)$
$C_1 \sqcup \dots \sqcup C_n$	Doctor $\sqcup$ Lawyer	$C_1(x) \vee \dots \vee C_n(x)$
$\neg C$	$\neg$ Male	$\neg C(x)$
$\{x_1\} \sqcup \dots \sqcup \{x_n\}$	{john} $\sqcup$ {mary}	$x = x_1 \vee \dots \vee x = x_n$
$\forall P.C$	$\forall$ hasChild.Doctor	$\forall y.P(x, y) \rightarrow C(y)$
$\exists P.C$	$\exists$ hasChild.Lawyer	$\exists y.P(x, y) \wedge C(y)$
$\leq_n P$	≤ 1 hasChild	$\exists \leq_n y.P(x, y)$

- The logic underpinning OWL, the standard (W3C recommendation) Ontology Web Language.
- A compromise between expressive power and computational complexity.
- Does not support: n-ary relations, identification, functional dependencies.



Mapping ORM to *SHOIN*

(Web) Information Systems



Mapping ORM to Description Logic

Why do we need this mapping for?

- Use ORM as a graphical notation for Ontology/description logic languages.

(The DL benefit from ORM)

- Reasoning on ORM schemes automatically.

(The ORM benefit from DL)

Reasoning Services

Examples of reasoning services:

- Satisfiability

To know whether a concept can be populated or not (e.g. because of some axioms contradicting each other)

$\text{SAT}(C, \mathcal{T})$ iff there is a model I of $\mathcal{T}$ with $C^I \neq \emptyset$

- Subsumption

To know whether a concept is subsuming another concept (e.g. to find unwanted or missing subsumptions)

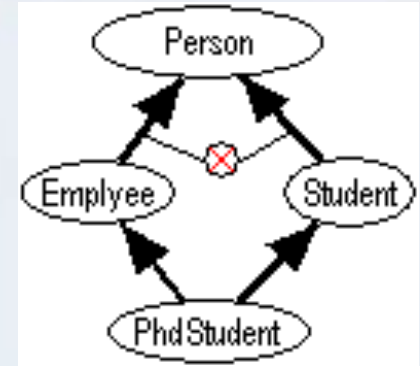
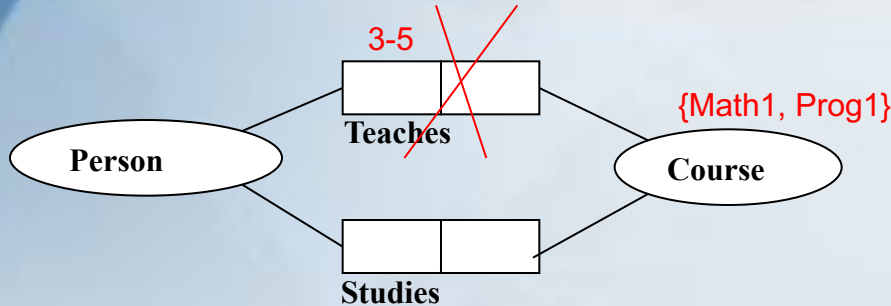
$\text{SUBS}(C, D, \mathcal{T})$ iff $C^I \subseteq D^I$ for all model I of $\mathcal{T}$

- Redundancies

To know whether two concepts are equal (e.g. to find out redundancies)

$\text{EQUIV}(C, D, \mathcal{T})$ iff $C^I = D^I$ for all model I of $\mathcal{T}$

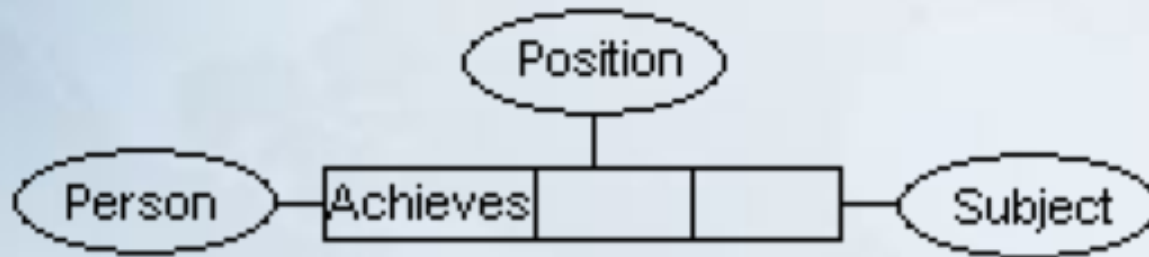
Reasoning on ORM Schemes



- Schema satisfiability: A schema is satisfiable if and only if there is at least one concept in the schema that can be populated. ➔ **Weak satisfiability**
 - Concept satisfiability: A schema is satisfiable if and only if all concepts in the schema can be populated.
 - Role satisfiability: A schema is satisfiable if and only if all roles in the schema can be populated. ➔ **Strong satisfiability**
- ➔ Concept satisfiability implies schema satisfiability.
- ➔ Role satisfiability implies concept satisfiability.

Mapping ORM to *DLR* and *SHOIN*

n-ary relations:

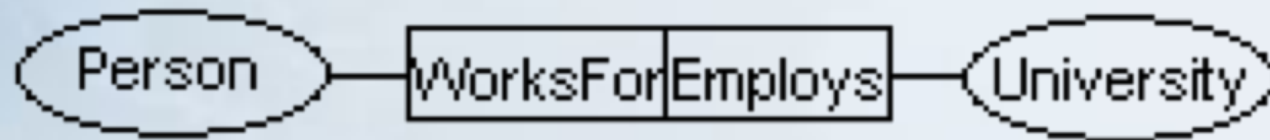


SHOIN

-- No Support --

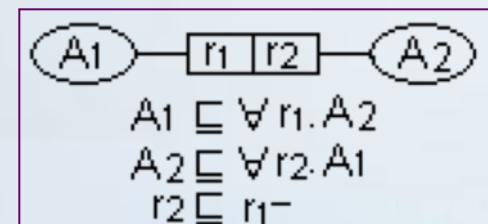
Mapping ORM to $\mathcal{DLR}$ and $\mathcal{SHOIN}$

Binary relations:



$\mathcal{SHOIN}$

$Person \sqsubseteq \forall WorksFor. University$
 $University \sqsubseteq \forall Employs. Person$
 $Employs \sqsubseteq WorksFor^{-}$



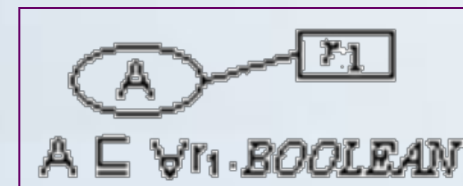
Mapping ORM to *DLR* and *SHOIN*

Unary roles:



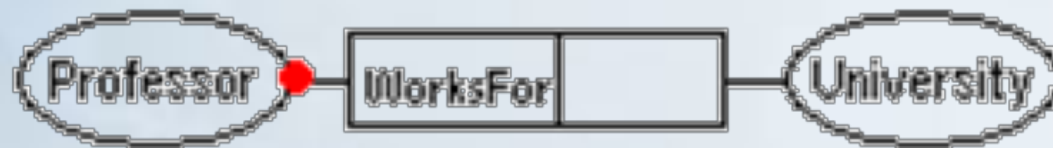
SHOIN

$Person \sqsubseteq \forall Smokes. \text{BOOLEAN}$



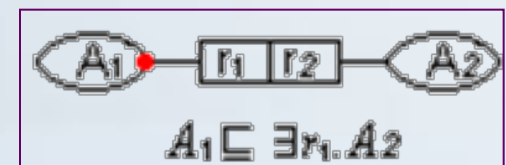
Mapping ORM to $\mathcal{DLR}$ and $\mathcal{SHOIN}$

Role Mandatory:



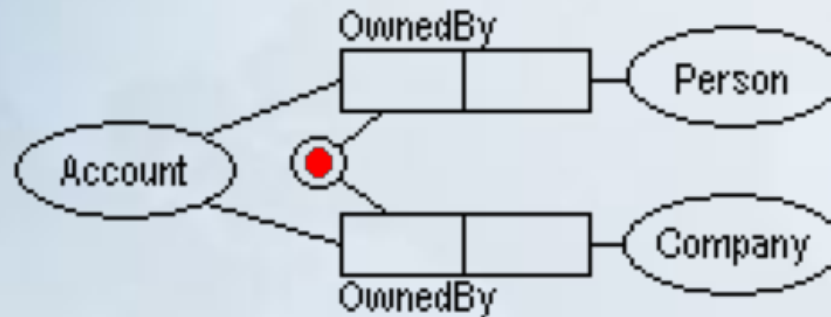
$\mathcal{SHOIN}$

$\text{Professor} \sqsubseteq \exists \text{WorksFor}.\text{University}$



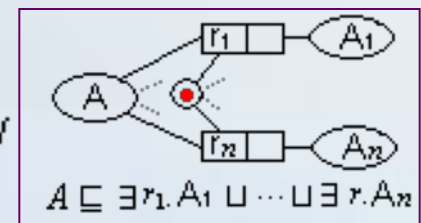
Mapping ORM to $\mathcal{DLR}$ and $\mathcal{SHOIN}$

Disjunctive Mandatory:



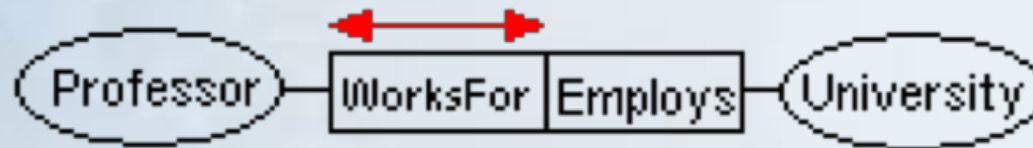
$\mathcal{SHOIN}$

$Account \sqsubseteq \exists OwnedBy_1. Person \sqcup \exists OwnedBy_2. Company$



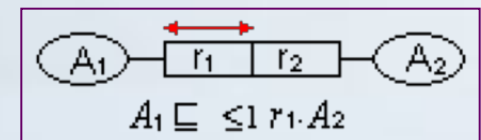
Mapping ORM to $\mathcal{DLR}$ and $\mathcal{SHOIN}$

Role Uniqueness:



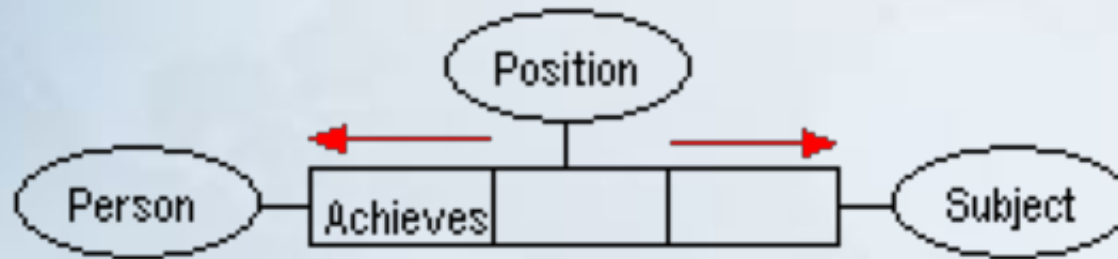
$\mathcal{SHOIN}$

$\text{Professor} \sqsubseteq \leq 1 \text{ WorksFor} . \text{University}$



Mapping ORM to *DLR* and *SHOIN*

Predicate Mandatory:

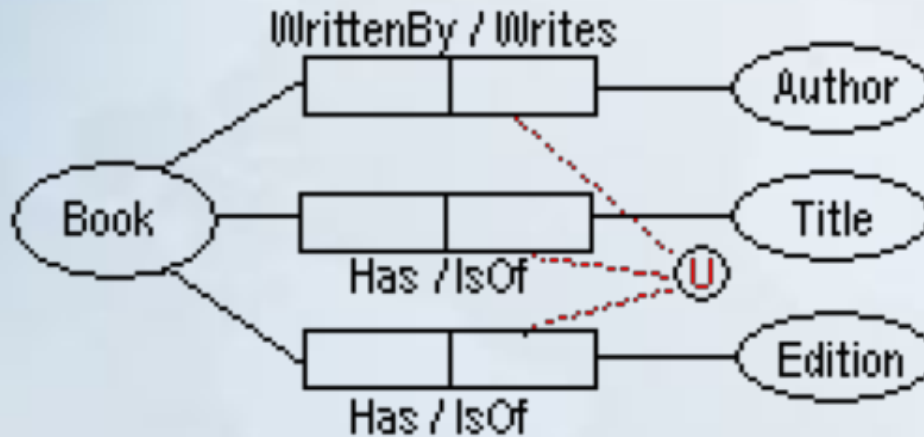


SHOIN

-- No Support --

Mapping ORM to *DLR* and *SHOIN*

External Mandatory:



SHOIN

-- No Support --

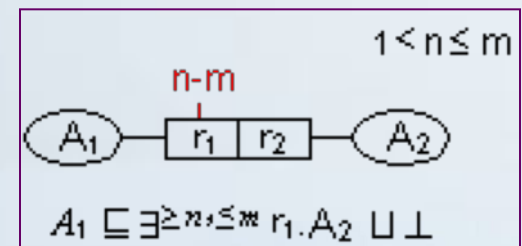
Mapping ORM to $\mathcal{DLR}$ and $\mathcal{SHOIN}$

Role Frequency:



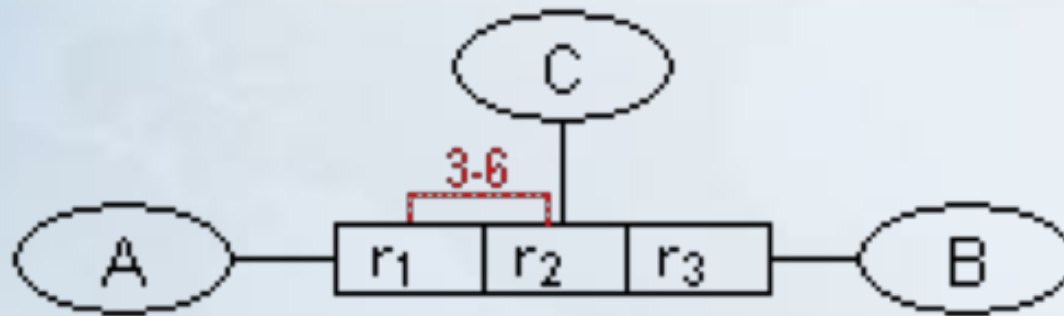
$\mathcal{SHOIN}$

$$Car \sqsubseteq \exists^{\geq 3, \leq 4} HasPart.Wheel \sqcup \perp$$



Mapping ORM to *DLR* and *SHOIN*

Multiple-Role Frequency:

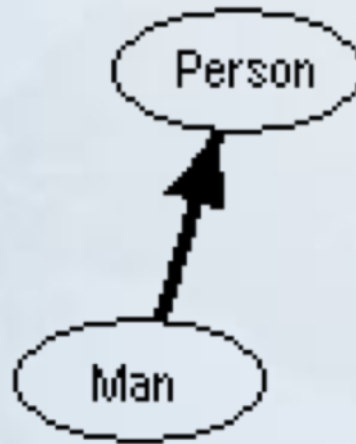


SHOIN

-- No Support --

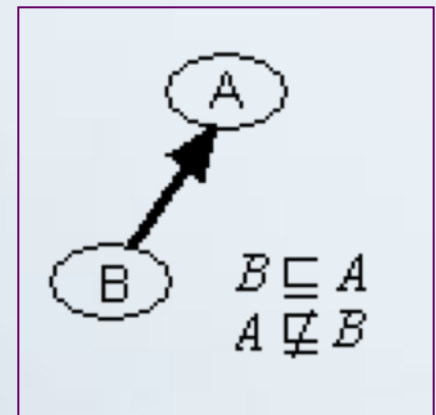
Mapping ORM to $\mathcal{DLR}$ and $\mathcal{SHOIN}$

Subtypes:



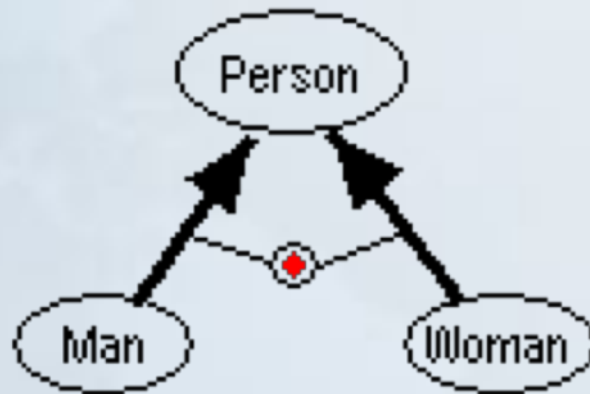
$\mathcal{SHOIN}$

$Man \sqsubseteq Person$
 $Person \not\sqsubseteq Man$



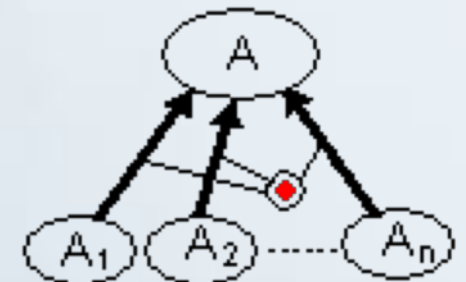
Mapping ORM to $\mathcal{DLR}$ and $\mathcal{SHOIN}$

Total Subtypes:



$\mathcal{SHOIN}$

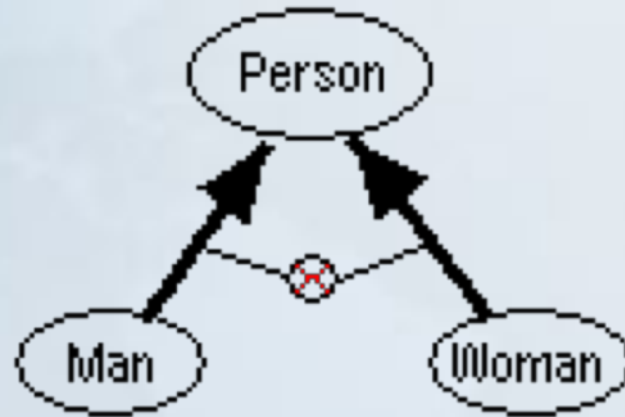
$Person \sqsubseteq Man \sqcup Woman$



$A \sqsubseteq A_1 \sqcup A_2 \sqcup \dots \sqcup A_n$

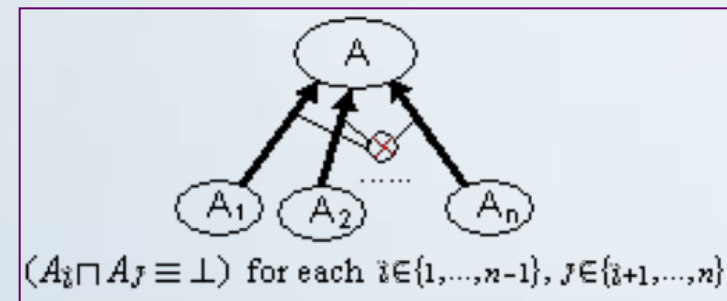
Mapping ORM to $\mathcal{DLR}$ and $\mathcal{SHOIN}$

Exclusive Subtypes:



$\mathcal{SHOIN}$

$$Man \sqcap Woman \equiv \perp$$



Mapping ORM to $\mathcal{DLR}$ and $\mathcal{SHOIN}$

Value Constraint:

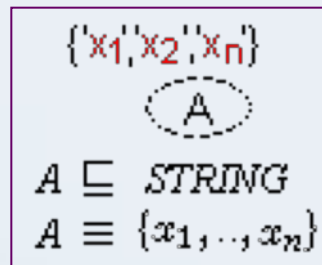
STRING:

$\{\text{Male}, \text{Female}\}$



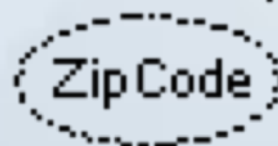
$\text{Gender} \sqsubseteq \text{STRING}$
 $\text{Gender} \equiv \{\text{Male}, \text{Female}\}$

$\mathcal{SHOIN}$



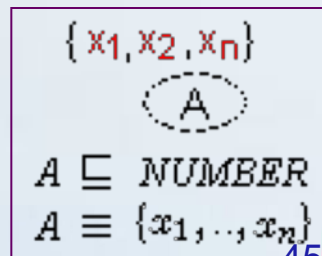
Number:

$\{1040, 1160\}$



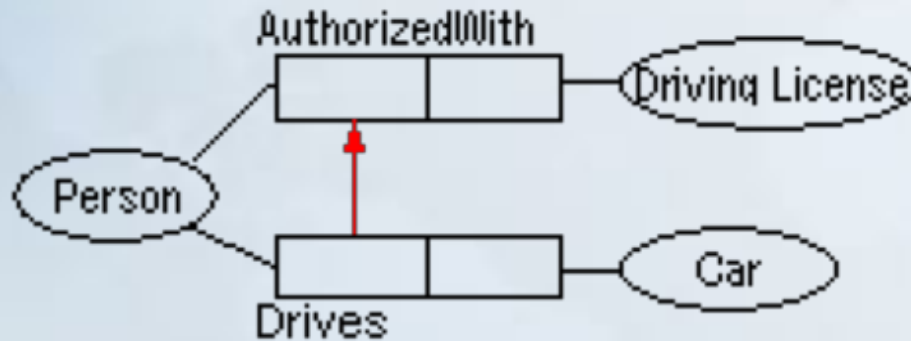
$\text{ZipCode} \sqsubseteq \text{NUMBER}$
 $\text{ZipCode} \equiv \{1040, 1160\}$

$\mathcal{SHOIN}$



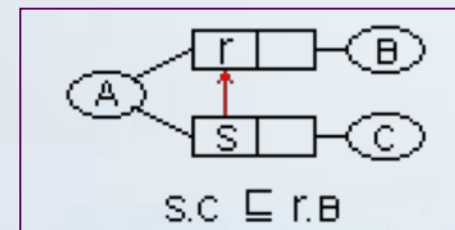
Mapping ORM to DLR and SHOIN

Role Subset Constraint:



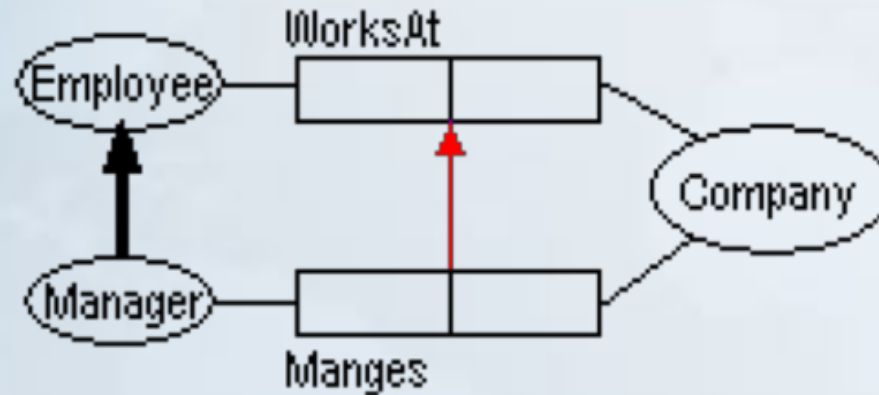
SHOIN

$Drives.Car \sqsubseteq AuthorizedWith.DrivingLicense$



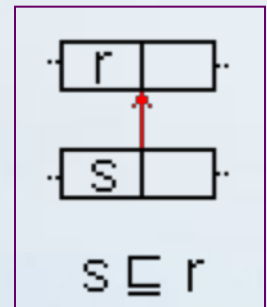
Mapping ORM to DLR and SHOIN

Predicate Subset Constraint:



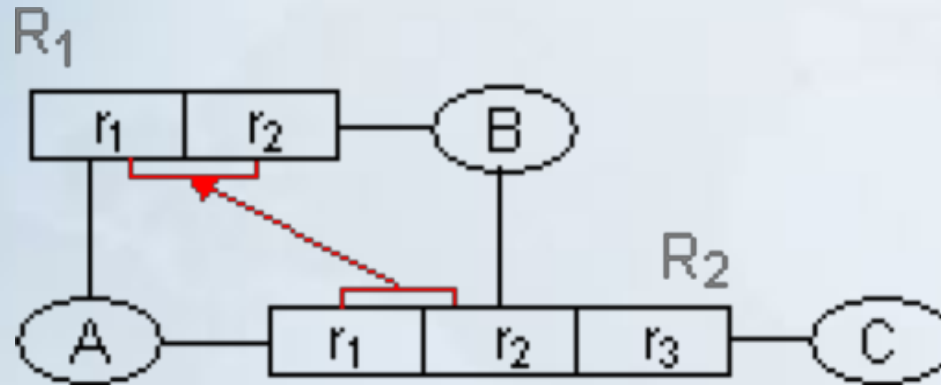
SHOIN

$\text{Manages} \sqsubseteq \text{WorksAt}$



Mapping ORM to DLR and SHOIN

Role-sequence Subset Constraint:

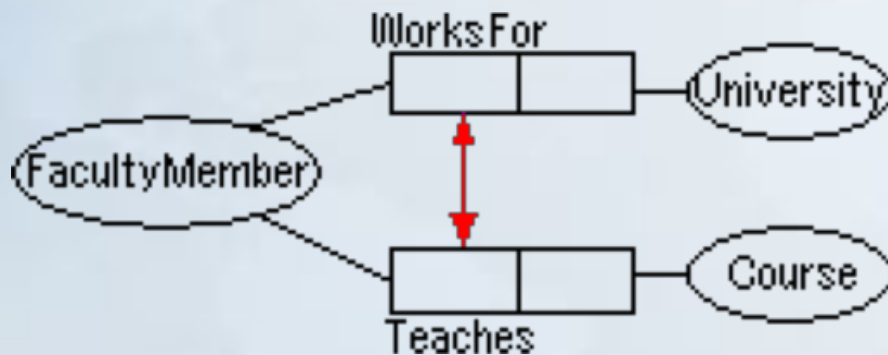


SHOIN

-- No Support --

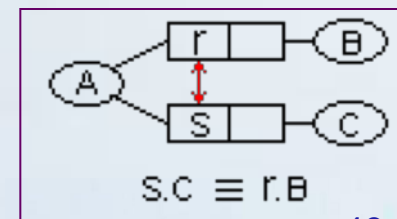
Mapping ORM to DLR and SHOIN

Role Equality Constraint:



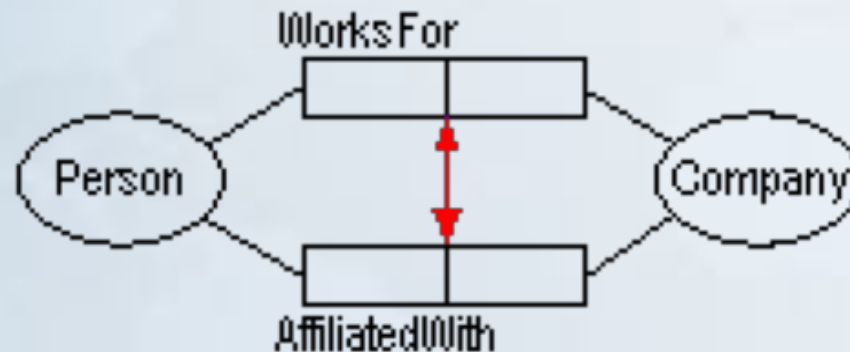
SHOIN

$WorksFor.University \equiv Teachs.Course$



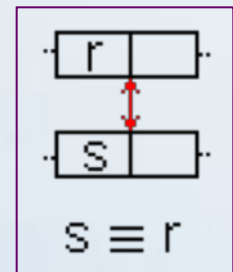
Mapping ORM to DLR and SHOIN

Predicate Equality Constraint:



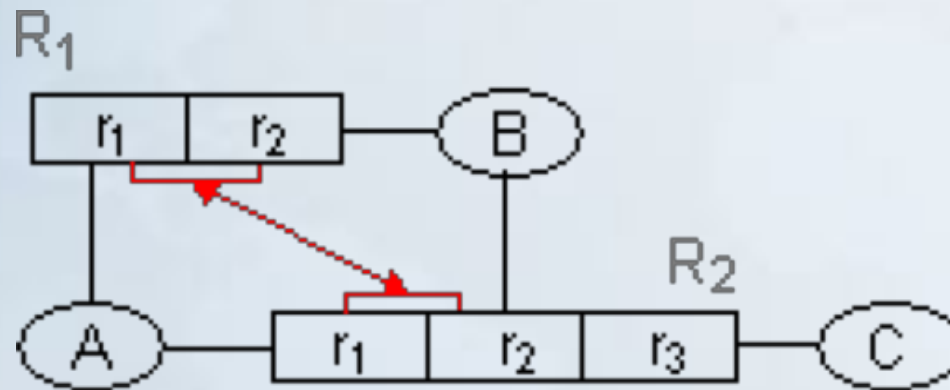
SHOIN

$WorksFor \equiv AffiliatedWith$



Mapping ORM to DLR and SHOIN

Role-sequence Equality Constraint:

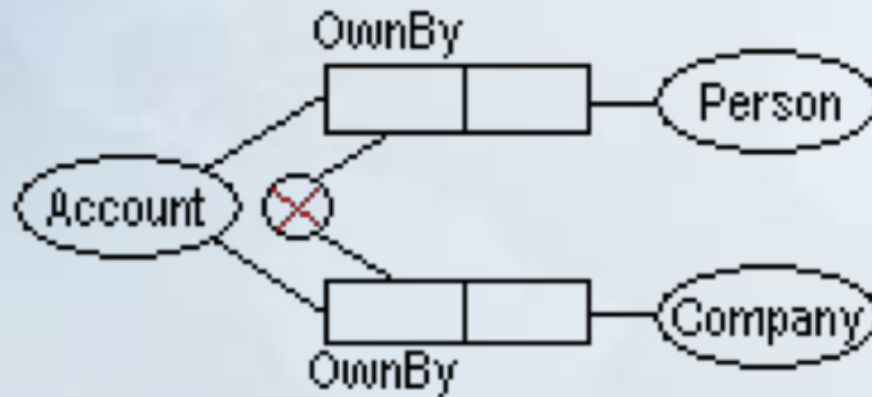


SHOIN

-- No Support --

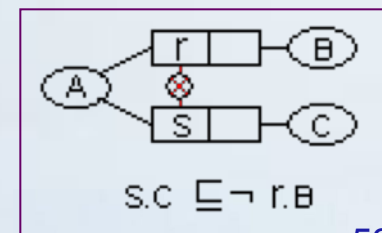
Mapping ORM to DLR and SHOIN

Role exclusion Constraint:



SHOIN

$OwnedBy.Person \sqsubseteq \neg OwnedBy.Account$



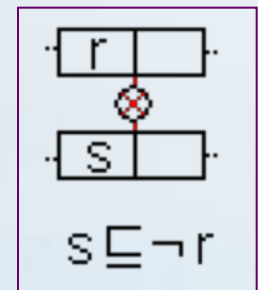
Mapping ORM to DLR and SHOIN

Predicate Exclusion Constraint:



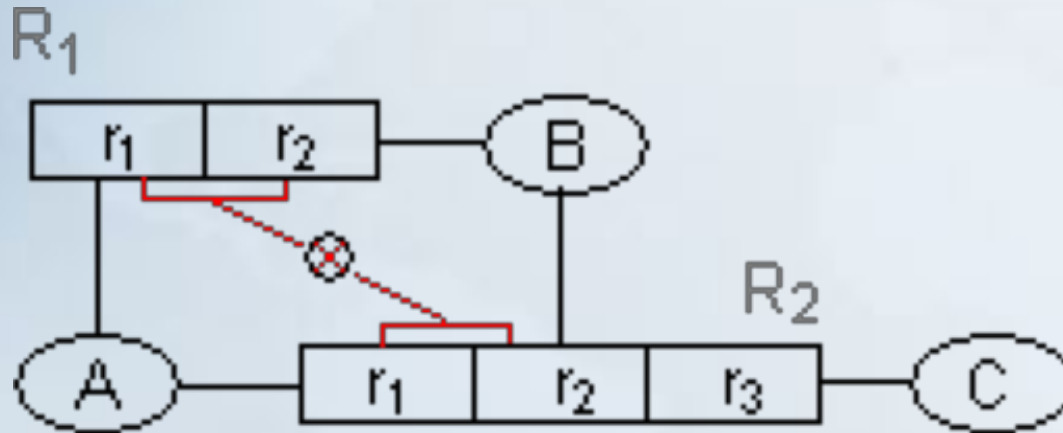
SHOIN

$Writes \sqsubseteq \neg Reviews$



Mapping ORM to DLR and SHOIN

Role- sequence Exclusion Constraint:

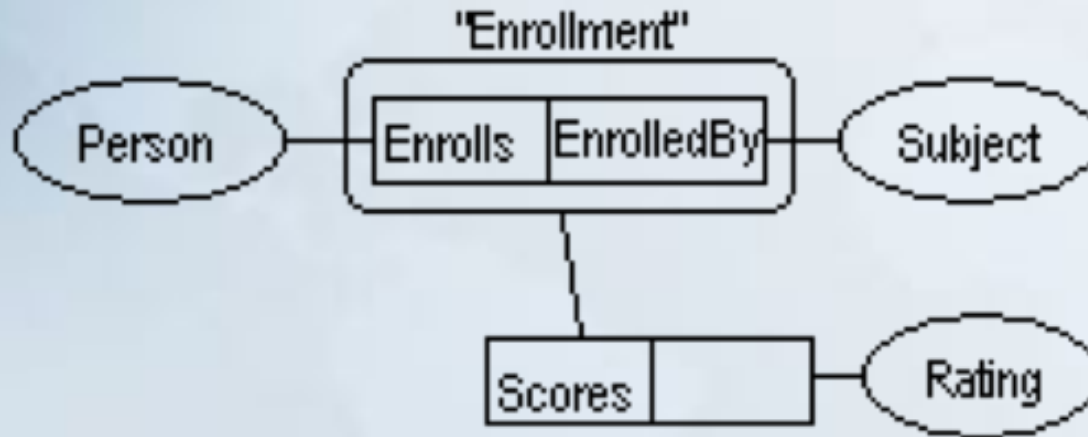


SHOIN

-- No Support --

Mapping ORM to *DLR* and *SHOIN*

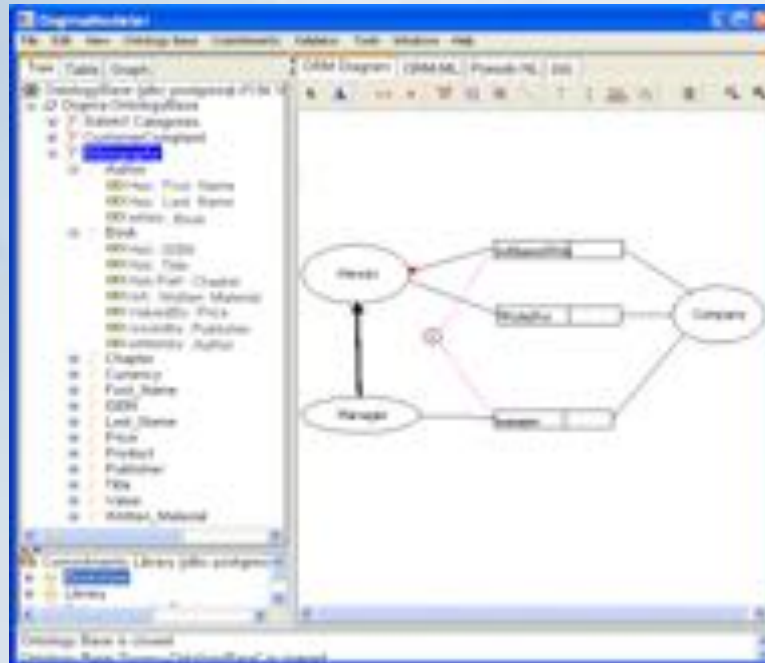
Objectification:



SHOIN

-- No Support --

DogmaModeler



- DogmaModeler is an ontology engineering tool
- It uses ORM as a graphical notation
- It uses Racer as a background reasoning engine

Other functionalities of DogmaModeler:

- Verbalization of ORM into 11 human languages.
- Modularization of auto composition of ORM schemes
-

Home Work

- Map your ORM model (project 3) into description logic.
- Reason about your ORM model using Racer.
- ➔MS Word file containing:
 1. Your ORM model
 2. Map ORM into DL
 3. Map ORM into DIG and load it to Racer
 4. Screen shots of Racer results, that (a)your model is satisfiable, (b) your model is unsatisfiable (make something wrong)

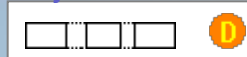
The next Lecture assumes that you know **XML**, if not please read about it.



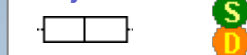
Conclusions and Discussion

ORM Constructs and constraints

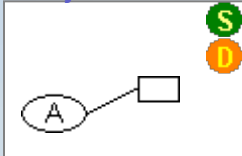
n-ary Relations



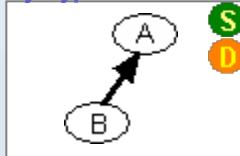
Binary Relations



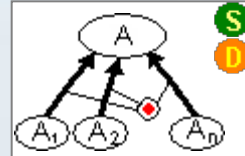
Unary Relations



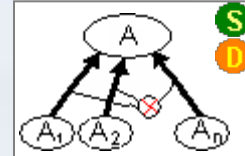
Sybtpe



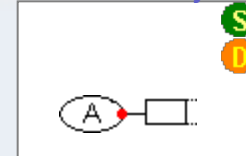
Total



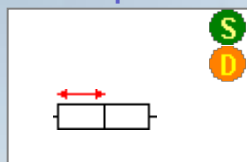
Exclusive



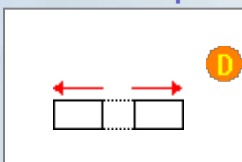
Role Mandatory



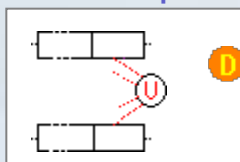
Role Uniqueness



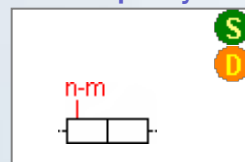
Predicate Uniqueness



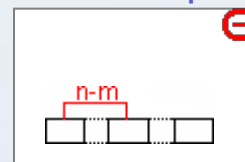
External Uniqueness



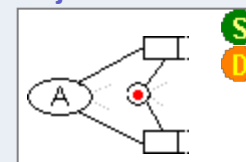
Role Frequency



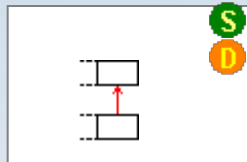
Multi Role Frequency



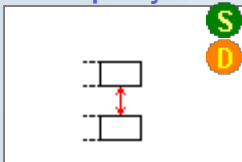
Disjunctive Mandatory



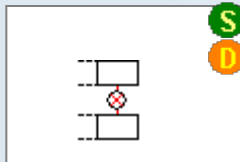
Role subset



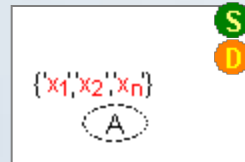
Role Equality



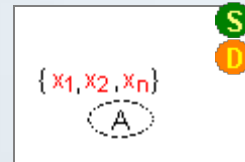
Role Exclusion



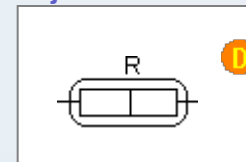
STRING Value



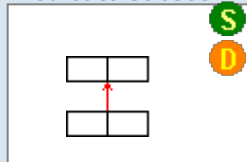
NUMBER Value



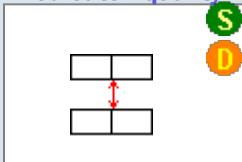
Objectification



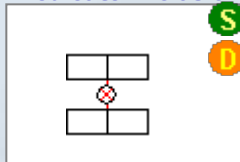
Predicate Subset



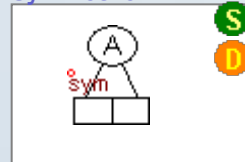
Predicate Equality



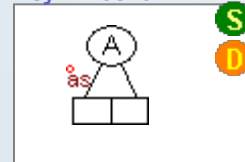
Predicate Exclusion



Symmetric



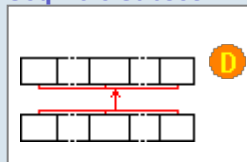
Asymmetric



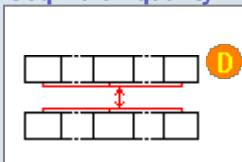
Anti Symmetric



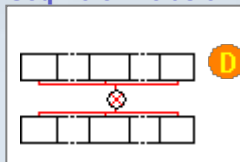
Seq-Role Subset



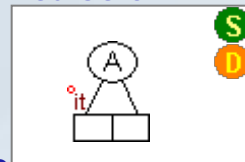
Seq-Role Equality



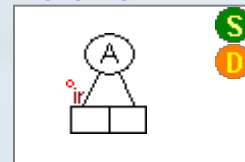
Seq-Role Exclusion



Intransitive



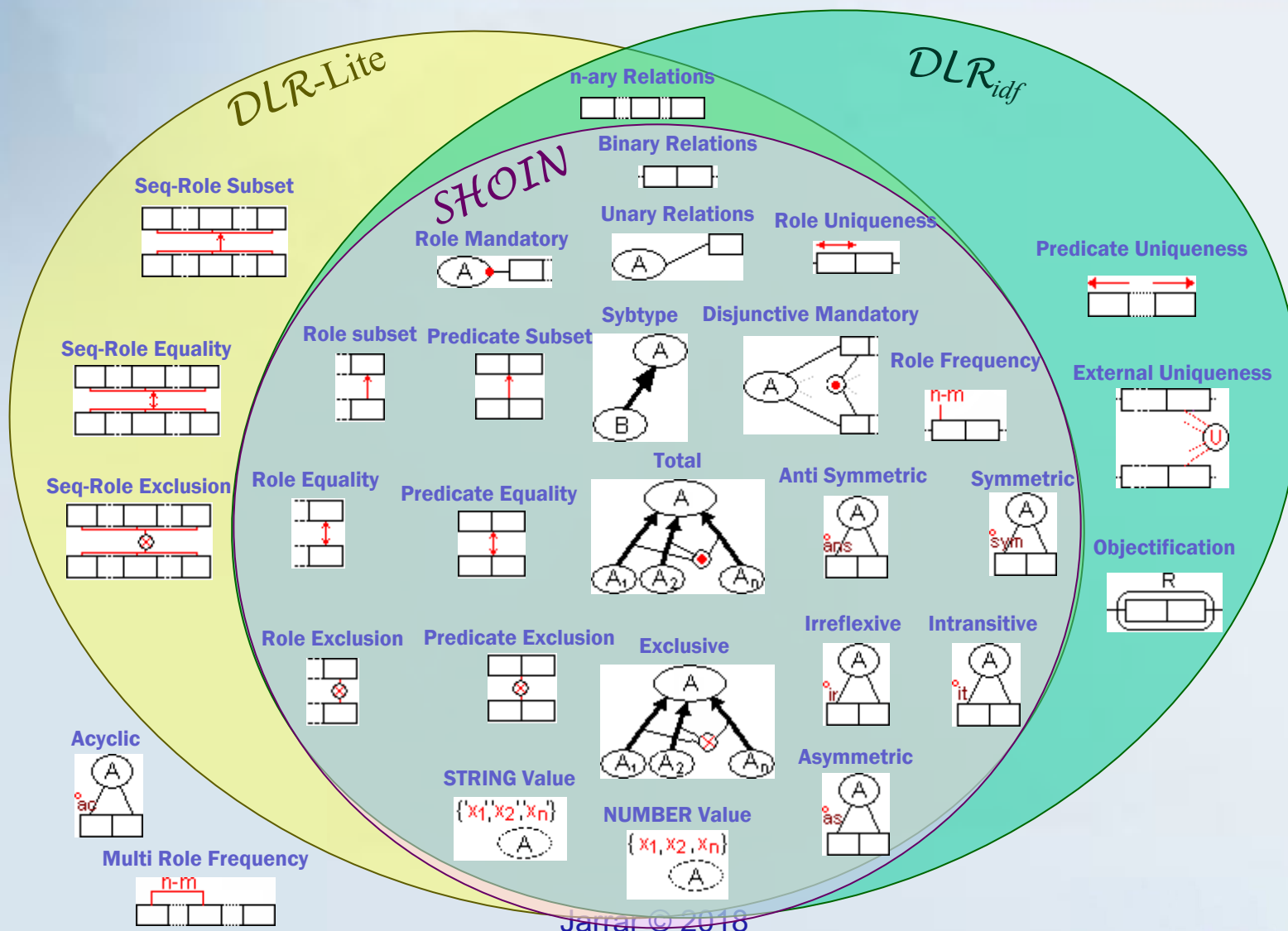
Irreflexive



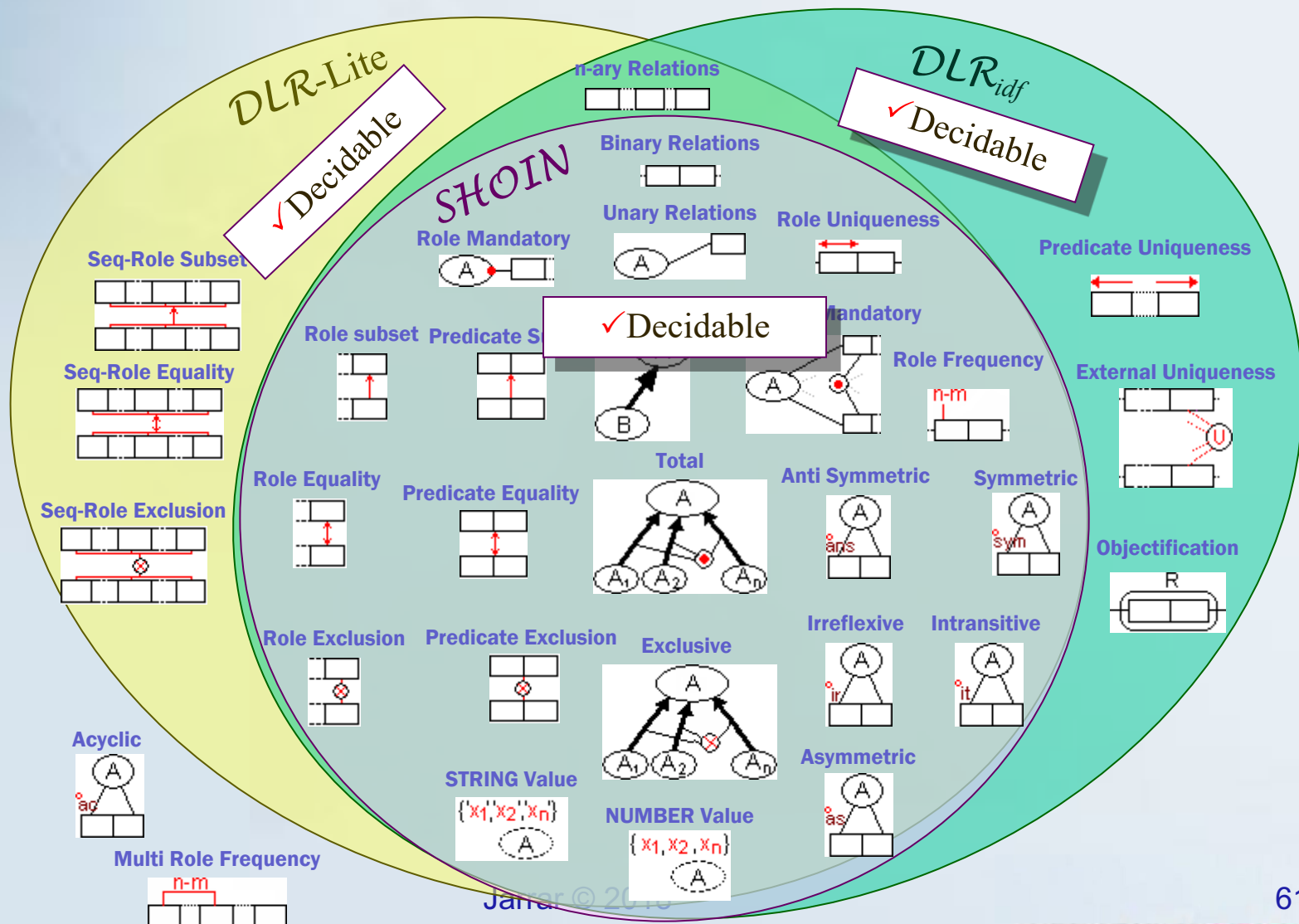
Acyclic

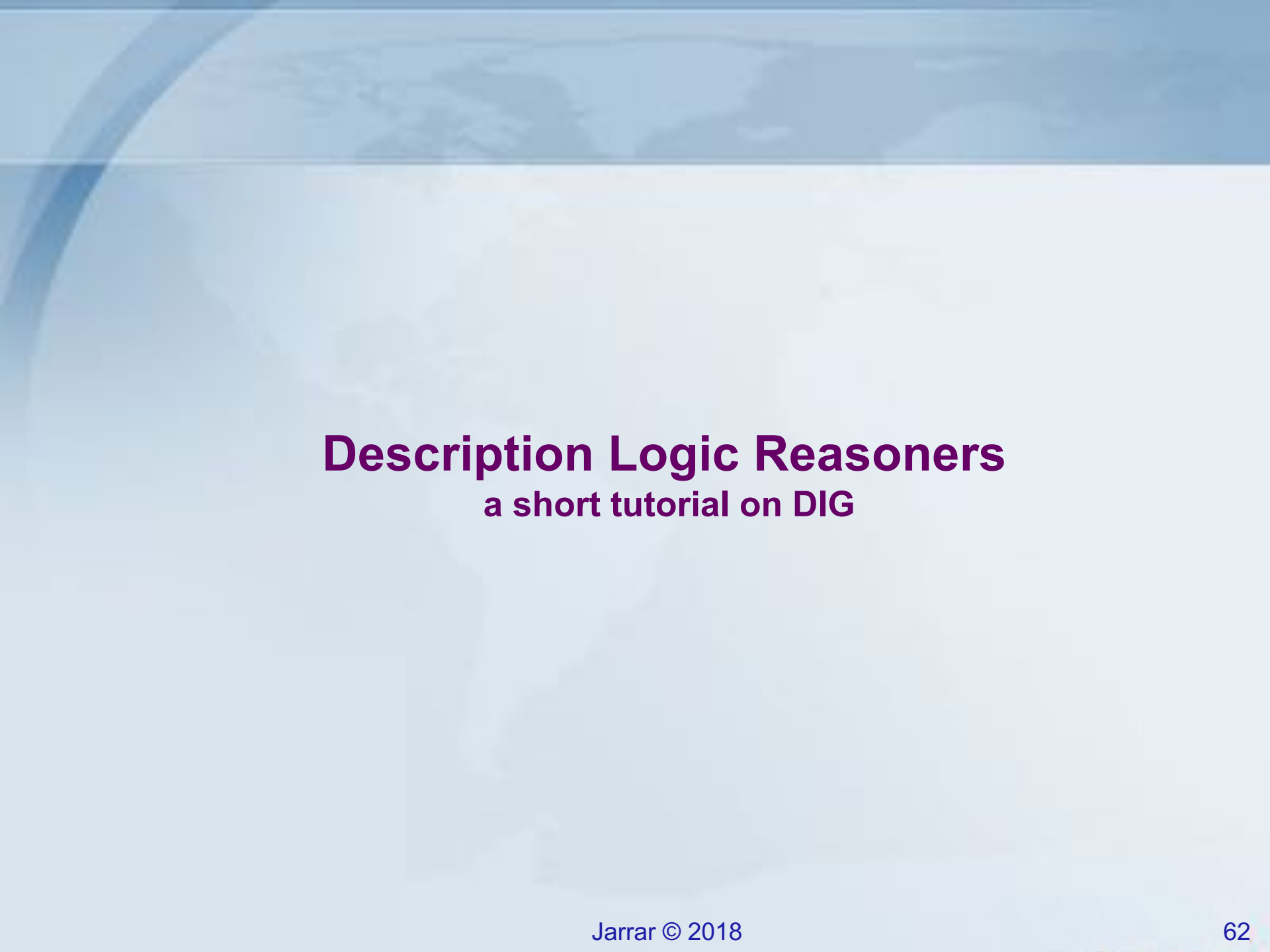


Lesson 1: what is mapped to what



Lesson 2: Decidable fragments





Description Logic Reasoners

a short tutorial on DIG

Description Logic Reasoners

- For example:



- They offer reasoning services for multiple TBoxes and ABoxes.
- They run as background reasoning engines.
- They understand DIG, which is a simple protocol (based on HTTP) along with an XML Schema.
- **Example:** $Student \sqsubseteq Person$

```
<impliesc>
  <catom name="Student"/>
  <catom name="Person"/>
</impliesc>
```

DIG Interface (<http://dig.sourceforge.net/>)



S. Bechhofer: The DIG Description Logic Interface: DIG/1.1

<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.104.3837&rep=rep1&type=pdf>

DIG Interface (<http://dig.sourceforge.net/>)



S. Bechhofer: The DIG Description Logic Interface: DIG/1.1

<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.104.3837&rep=rep1&type=pdf>

DIG Protocol

- DIG is only an XML schema for a description logic along with ask/tell functionality
- You write a new Knowledge base (using the DIG XML syntax), and send it to Racer using the TELL functionality.
- You can then write you Query/Question (using the DIG, XML syntax), and send it to Racer using the ASK functionality.
- You may communicate with the Racer through HTTP or SOAP.

Create e a new Knowledge Base

The newKB Message

```
<?xml version="1.0" encoding="UTF-8"?>
<newKB
xmlns="http://dl.kr.org/dig/2003/02/lang"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://dl.kr.org/dig/2003/02/lang
http://dl-web.man.ac.uk/dig/2003/02/dig.xsd"/>
```

The Response Message

```
<?xml version="1.0" encoding="UTF-8"?>
<response
xmlns="http://dl.kr.org/dig/2003/02/lang"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
xsi:schemaLocation="http://dl.kr.org/dig/2003/02/lang
http://dl-web.man.ac.uk/dig/2003/02/dig.xsd">
<kb uri="urn:uuid:abcdef01-234-1234-12345689ab"/>
```

This URI should then be used during TELL and ASK requests made against the knowledge base

Tell Syntax

A TELL request must contain in its body a tells element, which itself consists of a number of tell statements.

Example: $\text{Driver} \sqsubseteq \text{Person} \sqcap \exists \text{Drives.Vehicle}$

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<tells
  xmlns="http://dl.kr.org/dig/2003/02/lang"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://dl.kr.org/dig/2003/02/lang
http://dl-web.man.ac.uk/dig/2003/02/dig.xsd"
  uri="urn:uuid:abcdefgh-1234-1234-12345689ab">
  <defconcept name="driver"/>
  <equalc>
    <catom name="driver"/>
    <and>
      <catom name="person"/>
      <some>
        <ratom name="drives"/>
        <catom name="vehicle"/>
      </some>
    </and>
  </equalc>
  <defconcept name="person"/>
  <defconcept name="vehicle"/>
  <defrole name="drives"/>
</tells>
```

Jarrar © 2018

Tell Syntax

Tell Language	
Primitive Concept Introduction	<code><defconcept name="CN"/></code> <code><defrole name="CN"/></code> <code><deffeature name="CN"/></code> <code><defattribute name="CN"/></code> <code><defindividual name="CN"/></code>
Concept Axioms	<code><impliesc>C1 C2</impliesc></code> <code><equalc>C1 C2</equalc></code> <code><disjoint>C1... Cn</disjoint></code>
Role Axioms	<code><impliesr>R1 R2</impliesr></code> <code><equalr>R1 R2</equalr></code> <code><domain>R E</domain></code> <code><range>R E</range></code> <code><rangeint>R</rangeint></code> <code><rangestring>R</rangestring></code> <code><transitive>R</transitive></code> <code><functional>R</functional></code>
Individual Axioms	<code><instanceof>I C</instanceof></code> <code><related>I1 R I2</related></code> <code><value>I A V</value></code>

Concept Language	
Primitive Concepts	<code><top/></code> <code><bottom/></code> <code><catom name="CN"/></code>
Boolean Operators	<code><and>E1... En</and></code> <code><or>E1... En</or></code> <code><not>E</not></code>
Property Restrictions	<code><some>R E</some></code> <code><all>R E</all></code> <code><atmost num="n">R E</atmost></code> <code><atleast num="n">R E</atleast></code> <code><iset>I1... In</iset></code>
Concrete Domain Expressions	<code><defined>A</defined></code> <code><stringmin val="s">A</stringmin></code> <code><stringmax val="s">A</stringmax></code> <code><stringequals val="s">A</stringequals></code> <code><stringrange min="s" max="t">A</stringrange></code> <code><intmin val="i">A</intmin></code> <code><intmax val="i">A</intmax></code> <code><intequals val="i">A</intequals></code> <code><intrange min="i" max="j">A</intrange></code>
Role Expressions	<code><ratom name="CN"/></code> <code><feature name="CN"/></code> <code><inverse>R</inverse></code> <code><attribute name="CN"/></code> <code><chain>F1... FN A</chain></code> Individuals <code><individual name="CN"/></code>

Ask Syntax

- An ASK request must contain in its body an asks element.
- Multiple queries in one request is possible.



Ask Syntax

Ask Language	
Primitive Concept Retrieval	<code><allConceptNames/></code> <code><allRoleNames/></code> <code><allIndividuals/></code>
Satisfiability	<code><satisfiable>C</satisfiable></code> <code><subsumes>C1 C2</subsumes></code> <code><disjoint>C1 C2</disjoint></code>
Concept Hierarchy	<code><parents>C</parents></code> <code><children>C</children></code> <code><ancestors>C</ancestors></code> <code><descendants>C</descendants></code> <code><equivalents>C</equivalents></code>
Role Hierarchy	<code><rparents>R</rparents></code> <code><rchildren>R</rchildren></code> <code><rancestors>R</rancestors></code> <code><rdescendants>R</rdescendants></code>
Individual Queries	<code><instances>C</instances></code> <code><types>I</types></code> <code><instance>I C</instance></code> <code><roleFillers>I R</roleFillers></code> <code><relatedIndividuals>R</relatedIndividuals></code>

References

1. Mustafa Jarrar: Mapping ORM Into The SHOIN/OWL Description Logic- Towards A Methodological And Expressive Graphical Notation For Ontology Engineering . In OTM 2007 workshops: Proceedings of the International Workshop on Object-Role Modeling (ORM'07). Pages (729-741), LNCS 4805, Springer. ISBN: 9783540768890. Portugal. November, 2007
2. Mustafa Jarrar: Towards Automated Reasoning On ORM Schemes. -Mapping ORM Into The DLR_idf Description Logic. In proceedings of the 26th International Conference on Conceptual Modeling (ER 2007). Pages (181-197). LNCS 4801, Springer. Auckland, New Zealand. ISBN 9783540755623. November 2007
3. Mustafa Jarrar and Robert Meersman: Ontology Engineering -The DOGMA Approach. Book Chapter in "Advances in Web Semantics I". Chapter 3. Pages 7-34. LNCS 4891, Springer.ISBN:978-3540897835. (2008).
4. Mustafa Jarrar, Anton Deik, Bilal Faraj: Ontology-Based Data And Process Governance Framework -The Case Of E-Government Interoperability In Palestine . In pre-proceedings of the IFIP International Symposium on Data-Driven Process Discovery and Analysis (SIMPDA'11). Pages(83-98). ISBN 978-88-903120-2-1. Campione, Italy. June 30, 2011.
5. Mustafa Jarrar and Stijn Heymans: Unsatisfiability Reasoning In ORM Conceptual Schemes. In Current Trends in Database Technology - EDBT 2006: Proceeding of the IFIP-2.6 International Conference on Semantics of a Networked. Pages (517-534). LNCS 4254, Springer. Munich, Germany. ISBN: 3540467882. March 2006.
6. Mustafa Jarrar and Stijn Heymans: Towards Pattern-Based Reasoning For Friendly Ontology Debugging . Journal of Artificial Intelligence Tools. Volume 17. No.4. World Scientific Publishing. August 2008.
7. Mustafa Jarrar, Maria Keet, and Paolo Dongilli: Multilingual Verbalization Of ORM Conceptual Models And Axiomatized Ontologies. Technical report. STARLab, Vrije Universiteit Brussel, February 2006.
8. Sergey Lukichev and Mustafa Jarrar: Graphical Notations For Rule Modeling . Book chapter in "Handbook of Research on Emerging Rule-Based Languages and Technologies". IGI Global. ISBN:1-60566-402-2. (2009)
9. Mustafa Jarrar: Modularization And Automatic Composition Of Object-Role Modeling (ORM) Schemes .OTM 2005 Workshops: Proceedings of the Object-Role Modeling (ORM'05). Pages (613-625). LNCS 3762, Springer. ISBN: 3540297391. 2005.
10. Mustafa Jarrar: Towards Methodological Principles For Ontology Engineering. PhD Thesis. Vrije Universiteit Brussel. (May 2005)
11. Mustafa Jarrar, Jan Demey, and Robert Meersman: On Using Conceptual Data Modeling For Ontology Engineering . Journal on Data Semantics, Special issue on "Best papers from the ER/ODBASE/COOPIS 2002 Conferences". LNCS 2800. No 1. Springer. 2003.
12. Jan Demey, Mustafa Jarrar, and Robert Meersman: A Markup Language For ORM Business Rules . Proceedings of the International Workshop on Rule Markup Languages for Business Rules on the Semantic Web (RuleML 2002). Pages(107-128). Volume 60. CEUR Workshop Proceedings. ISSN 1613-0073. June 2002
13. Mustafa Jarrar: Towards Effectiveness And Transparency In E-Business Transactions, An Ontology For Customer Complaint Management . A book chapter in "Semantic Web Methodologies for E-Business Applications". chapter 7. IGI Global. (2008)
14. Mustafa Jarrar: ORM Markup Language, Version 3 . Technical Report. STAR Lab, Vrije Universiteit Brussel, Belgium. January 2007